

GAMES101 Assignment 3 — Rasterization & Shading

Overview

This project implements multiple real-time rendering techniques based on the GAMES101 Assignment 3 framework.

Implemented features include:

- Perspective projection matrix
- Triangle rasterization with barycentric interpolation
- Blinn-Phong shading
- Texture mapping
- Bump Mapping
- Displacement Mapping
- Bilinear Texture Interpolation (Bonus)

The renderer supports multiple fragment shaders and demonstrates how different shading and texture sampling techniques affect the final rendered result.

Features

1. Projection Matrix

Implemented a perspective projection matrix in `get_projection_matrix()`.

Key concepts:

- Perspective-to-orthographic transformation
 - Orthographic scaling and translation
 - Perspective divide
-

2. Triangle Rasterization

Implemented interpolation inside:

rasterize_triangle()

Interpolated attributes:

- Vertex color
- Surface normals
- Texture coordinates
- View-space positions

Used barycentric coordinates for perspective-correct interpolation.

3. Blinn-Phong Shading

Implemented:

- Ambient lighting
- Diffuse lighting
- Specular highlights

Core equation:

$\text{FinalColor} = \text{Ambient} + \text{Diffuse} + \text{Specular}$

4. Texture Mapping

Implemented texture sampling using UV coordinates.

Texture color is used as:

- Diffuse coefficient (k_d)

Supported:

- Nearest Neighbor Sampling
 - Bilinear Texture Interpolation
-

5. Bump Mapping

Implemented tangent-space normal perturbation using a height map.

Main steps:

- Construct TBN matrix
- Compute height derivatives (dU , dV)
- Generate perturbed normal
- Transform tangent-space normal into view space

Bump mapping changes surface normals without modifying geometry.

6. Displacement Mapping

Extended bump mapping by additionally modifying surface position.

Implemented:

$$p = p + kn * n * h(u,v)$$

Unlike bump mapping, displacement mapping changes both:

- Surface normals
- Surface positions

This creates stronger geometric depth and lighting variation.

7. Bilinear Texture Interpolation (Bonus)

Implemented:

```
Vector3f getColorBilinear(float u, float v)
```

Bilinear interpolation samples the four surrounding texels and performs weighted interpolation to produce smoother texture transitions.

Comparison:

Nearest Neighbor

- Sharp edges
- Pixelated appearance
- Strong block artifacts

Bilinear Interpolation

- Smoother transitions
- Reduced blockiness
- Slightly blurrier textures

To better visualize the difference, low-resolution textures were used during testing.

Rendering Results

Implemented shaders:

- Normal Shader



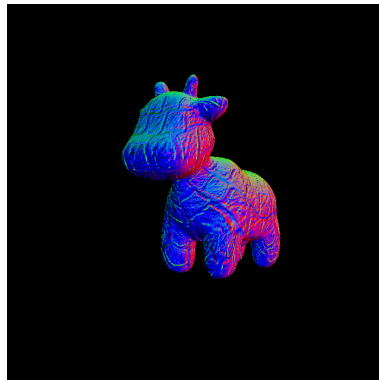
- Phong Shader



- Texture Shader



- Bump Shader



- Displacement Shader



Additional comparison:

- Nearest Neighbor vs Bilinear Texture Sampling



Technologies

- C++
 - Eigen
 - OpenCV
 - OBJ Loader
-

How to Run

Build the project:

```
cmake --build .
```

```
.\Debug\Rasterizer1.exe output.png normal
```

```
.\Debug\Rasterizer1.exe output.png phong
```

```
.\Debug\Rasterizer1.exe output.png texture
```

```
.\Debug\Rasterizer1.exe output.png bump
```

```
.\Debug\Rasterizer1.exe output.png displacement
```

Author

Developed as part of the GAMES101 Computer Graphics coursework.